

A.Y. 2025–2026

Software Engineering

for

AstraLog-HPC Project

RASD & DD

Antonello Anzalone
Gabriele Amodeo
Francesco Agosta

Version 2.0
April 2026

Contents

1	The project and project goals	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions/Acronyms	2
2	Requirement analysis	4
2.1	Relevant human and non-human actors	5
2.1.1	Human Actors	5
2.1.2	Non Human Actors	5
2.2	Scenarios and Use Cases	5
2.2.1	Scenarios	5
2.2.2	Use Cases	6
2.3	Domain assumptions	9
2.3.1	Configuration and Metadata	9
2.3.2	Data Nature and Temporal Logic	9
2.3.3	Execution Environment (HPC)	9
2.3.4	Universal Standards	10
2.4	Requirements	10
2.4.1	Functional requirements	10
	System Configuration & Initialization	10
	Telemetry Ingestion & Validation	10
	Monitoring Rule Evaluation	10
	Result Reporting & Logging	11
2.4.2	Non-functional requirements	11
3	Design	12
3.1	General Description of the Architecture	12
3.1.1	Architectural Components	13
3.1.2	Architectural Connectors	13
3.2	Sequence diagrams	14
3.2.1	Simple, Step-Difference, and Stateful Rule Evaluation	14
3.2.2	Logical Correlation Rule Evaluation	14
3.3	Critical Points and Design Decisions	17
3.3.1	Maximizing Performance through Shared-Memory Parallelism	17
3.3.2	Handling Correlation after the Parallel Phase	17
3.3.3	Modularity, Cohesion, and Defensive Boundaries	17
3.3.4	Information Hiding and I/O Bottleneck Management	17
3.4	Design-to-Implementation Deviations	18
3.5	Deployment Diagram	18
4	Work distribution and effort	20
4.1	Personal Contribution and Role in Requirements Analysis and System Design	20
5	Usage of AI	21

1 The project and project goals

1.1 Purpose

The project serves two main purposes. Academically, the objective is to practice software engineering concepts, including requirement engineering, architectural design, automated testing, containerization and the use of High-Performance Computing (HPC) clusters via SLURM. The goal is focused on "learning by doing" rather than just delivering working software.

In the context of the simulated scenario, the purpose is to respond to a "Call for Tenders" from the European Space Agency (ESA) by developing AstraLog-HPC, a software system designed to ensure spacecraft safety by identifying anomalies in aircraft subsystems.

1.2 Scope

The overall project is divided into two phases: creating a requirement analysis and design document (Phase 1), and building a Continuous Integration/Continuous Deployment (CI/CD) pipeline alongside either a provided implementation or a custom prototype built in C++ or Python (Phase 2).

The specific scope of the AstraLog-HPC software system includes:

- **Data Ingestion and Filtering:** Ingesting telemetry data batches from provided `.csv` files and filtering out corrupted data packets caused by transmission noise, specifically targeting malformed CSV rows, missing mandatory fields (schema errors) and invalid data types.
- **Rule Evaluation:** Accumulating valid packets into batches and applying four types of monitoring rules (Simple Absolute Threshold, Step Difference, Stateful Measurement Persistence, and Logical Correlation) defined in a `rules.json` file. This evaluation relies on a unified synchronous system clock where all measurements are equally spaced in time and perfectly aligned.
- **System Outputs:** The software must strictly adhere to specified output formats, appending nominal states to `valid_data.csv` and anomalous states (rule violations) to `alarms.log`.
- **Deployment Environment:** The system must be capable of running on classical machines/virtual machines as well as on an HPC cluster utilizing the SLURM job scheduler and Singularity containers.

1.3 Definitions/Acronyms

The key definitions and acronyms relevant to the project:

- **HPC:** High-Performance Computing.
- **ESA:** European Space Agency.

- **SLURM**: The job scheduler used to manage and execute jobs on the HPC cluster.
- **CI/CD**: Continuous Integration/Continuous Deployment. The automated pipelines required for building, testing, and releasing the project.
- **JSON**: JavaScript Object Notation. In this project, JSON is used both for representing telemetry packets embedded in the input `.csv` files and for defining the monitoring rules contained in `rules.json`.
- **YAML**: YAML Ain't Markup Language. In this project, YAML is used in the `sensors.yaml` configuration file to define the available sensors and their metadata, such as valid `sensor_id` values and sensor-related configuration information used during parsing and validation.
- **RASD**: Requirement Analysis and Specification Document. The culmination of the Requirement Engineering process containing the system's purpose, scope, use cases, models, and definitions.
- **DD**: Design Document. The artifact that captures the system architecture and detailed design decisions, including components, interfaces, data structures and deployment considerations, serving as a blueprint for implementation.
- **GenAI / LLMs**: Generative AI / Large Language Models (such as ChatGPT, Gemini and Claude), which can be used to assist the project under strict accountability and ethical guidelines.

2 Requirement analysis

The Machine (AstraLog-HPC)

This is the software-to-be developed to identify anomalies in aircraft subsystems. It includes the logic to receive streamed data, filter invalid packets, and evaluate four types of monitoring rules (simple, step difference, stateful, and correlation). It also encompasses the execution environment, such as the HPC cluster (Galileo 100) and the SLURM job scheduler.

The World (Environment)

This captures the part of the real world influenced by the Machine—namely, the aircraft subsystems under monitoring. Within this context, actors naturally emerge as the external entities that interact with AstraLog-HPC to accomplish specific objectives.

Classification of Phenomena

World Phenomena

These are events occurring entirely outside the system, such as the actual physical conditions of the aircraft hardware and electromagnetic interference that causes real-world data corruption, which the machine cannot observe directly.

Machine Phenomena

These are purely internal software events, including syntactic validation of incoming JSON packets, aggregation of data into local batches, execution of rule logic (simple, stateful, step-based, and correlation rules) and management of internal memory to track sequences of rule violations.

Shared Phenomena

These are events and data residing at the boundary between the environment (World) and the software (Machine).

- **World-Controlled**

- **Telemetry Stream:** A simulated sequence of JSON packets (timestamps, sensor IDs, values) representing spacecraft data, which our system ingests as static `.csv` batch files rather than a continuous live broadcast.
- **Configuration Inputs:** Defined sensor types and monitoring rules provided via `sensors.yaml` and `rules.json`.
- **HPC Scheduling:** Resource allocation and execution timing decided by the SLURM scheduler on the Galileo100 cluster.

- **Machine-Controlled**

- **System Outputs:** Generation of aggregated nominal data in `valid_data.csv` and specific anomaly entries in `alarms.log`.
- **Job Submission:** Initiation of the `job.sh` script and deployment of Singularity containers to the cluster environment.
- **Output Redirection:** Automated capturing of standard output and error logs into dedicated text files.

2.1 Relevant human and non-human actors

2.1.1 Human Actors

Ground Station Engineer: This single actor handles both system setup and ongoing monitoring. His responsibilities include:

- **Configuration:** Defining the types and instances of sensors in the `sensors.yaml` file, and setting the operational parameters—such as batch size—and the monitoring parameters (Simple Absolute Threshold, Step Difference, Stateful Measurement Persistence, and Logical Correlation rules) in the `rules.json` file.
- **Monitoring:** Reviewing the aggregated healthy spacecraft states in the `valid_data.csv` file and responding to rule violations and anomaly alerts generated in the `alarms.log` file.

2.1.2 Non Human Actors

Spacecraft / Sensors: The original external entities in the domain that continuously produce telemetry data (such as temperature, pressure, or voltage) as JSON packets simulated and provided through static `.csv` input files.

2.2 Scenarios and Use Cases

2.2.1 Scenarios

System Initialization

Scenario 1: *Successful System Configuration*

The Ground Station Engineer prepares the system by providing valid `sensors.yaml` and `rules.json` files. They launch AstraLog-HPC and specify a batch size of 100 packets. The software successfully parses the configuration, maps the sensor instances to their respective monitoring rules, and enters a "Ready" state to begin processing the provided telemetry files.

Telemetry Ingestion and Filtering

Scenario 2: *Data Ingestion from CSV Files*

The system reads telemetry data from the provided `.csv` files. AstraLog-HPC parses the records, extracting the timestamp, sensor ID, and numerical values for each aircraft subsystem. The software confirms each record matches the expected schema and appends the data to a local batch for analysis until the user-specified batch size is reached.

Scenario 3: *Filtering Corrupted Telemetry*

While processing the input data, the system encounters records with malformed syntax, missing mandatory fields, or type errors (e.g., a string where a number is expected). AstraLog-HPC's ingestion logic identifies these structural issues, discards the corrupted records, and continues processing the remaining valid data in the file.

Analysis and Outcome Reporting

Scenario 4: *Reporting a Healthy Spacecraft*

After accumulating a full batch, AstraLog-HPC evaluates all monitoring rules and determines that every measurement is within safety limits. The software appends a single aggregated line to `valid_data.csv`, documenting the timestamp and a `NOMINAL` status for all sensors. The Ground Station Engineer reviews the file to confirm the spacecraft's healthy state.

Scenario 5: *Alerting on a Simple Threshold Anomaly*

During rule evaluation, AstraLog-HPC processes a Simple Rule checking if a sensor exceeds a fixed threshold (e.g., `TEMP-01 > 50.0°C`). The system identifies a violation and logs a `MEDIUM` priority alert in `alarms.log`, detailing the timestamp, rule ID, and the violated sensor value.

Scenario 6: *Alerting on a Step-Difference Anomaly*

The system computes the signed variation between the current sensor value and the immediately preceding valid measurement as `current - previous`. If the configured Step Difference Rule operator is satisfied by this signed delta, the system immediately appends an anomaly entry to `alarms.log` for the Ground Station Engineer to review.

Scenario 7: *Monitoring Persistent Voltage Failure*

The software identifies a voltage drop below the required threshold. It monitors the violation through its internal tracking logic by evaluating the number of consecutive valid measurements in which the condition persists. On the final required continuous violation, the system triggers a Stateful Rule and logs the persistent anomaly in `alarms.log`.

Scenario 8: *Identifying Compound Risks*

At a single timestamp, the system identifies that both a high-temperature rule and a pressure-drop rule have been evaluated to be true. AstraLog-HPC evaluates a Logical Correlation Rule with an AND operator. Recognizing the combined risk, the system logs a high-priority alarm in `alarms.log` listing all involved sensors and values.

2.2.2 Use Cases

Use Case 1: System Configuration and Initialization

Name: *Configure and Launch Monitoring*

Participating Actors: Ground Station Engineer

Entry Condition: The `sensors.yaml` and `rules.json` files are present in the designated configuration directory.

Flow of Events:

1. The Ground Station Engineer accesses the AstraLog-HPC command-line interface.

2. The Engineer provides the file paths for the sensor definitions (`sensors.yaml`) and the monitoring rules (`rules.json`).
3. The Engineer specifies the target batch size for telemetry processing.
4. The System parses the configuration files to map specific sensor instances to their allowed types and priorities.
5. The System establishes a connection to the telemetry data source.

Exit Condition: The System enters a "Ready" state and begins ingesting the telemetry data batch.

Special Requirements: The system must provide feedback confirming that all rules have been successfully loaded before starting ingestion.

Use Case 2: Telemetry Ingestion and Validation

Name: *Ingest and Validate Telemetry*

Participating Actors: Spacecraft/Sensor

Entry Condition: AstraLog-HPC is in a "Ready" state and the connection to the data source is active.

Flow of Events:

1. The System reads consecutive telemetry data rows (containing JSON strings) from the provided `.csv` input file.
2. The System parses each packet.
3. The System performs a structural validation check on the packet.
4. If the record is valid, the System extracts the `timestamp`, `sensor_id`, and `value`, while also validating the presence of the CSV `priority` field.
5. The System appends the valid record to a local temporary batch storage.
6. The System repeats this process until the batch reaches the size specified by the Engineer.

Exit Condition: The local batch storage contains the exact number of valid packets required to trigger an analysis.

Exceptions:

- Malformed CSV Rows: The System identifies truncated strings or invalid syntax; it discards the packet and logs a syntax error.
- Schema Errors: The System detects missing mandatory fields (e.g., no `sensor_id`); it discards the packet.
- Type Errors: The System identifies invalid data types (e.g., a string in the `value` field); it ignores the record.

Special Requirements: Filtering must occur immediately upon reception to prevent corrupted data from entering the analysis batch.

Use Case 3: Batch Analysis and Anomaly Reporting

Name: *Analyze Batch and Detect Anomalies*

Participating Actors: Ground Station Engineer

Entry Condition: The local batch file has reached the Engineer-specified size.

Flow of Events:

1. The System reads the accumulated batch of valid telemetry records.
2. The System sorts and prioritizes the monitoring rules (High > Medium > Low).
3. For each timestamp T_n , the System evaluates:
 - Simple Rules: Checks if the absolute value exceeds a fixed threshold.
 - Step Difference Rules: Computes the signed delta between the current value at T_n and the previous value at T_{n-1} , then applies the configured operator directly to that signed delta.
 - Stateful Rules: Updates the count of consecutive violations stored in memory.
 - Logical Correlation Rules: Combines Boolean outcomes of multiple rules using AND/OR logic.
4. **Outcome A (Nominal)**: If no rules trigger, the System appends a single aggregated line to `valid_data.csv`.
5. **Outcome B (Anomalous)**: If any rule evaluates to `TRUE`, the System appends one entry per violation to `alarms.log`, listing all involved sensors and values.
6. The System clears the local batch file to prepare for the next ingestion cycle.

Exit Condition: All records in the batch are processed, and the output files are updated on disk.

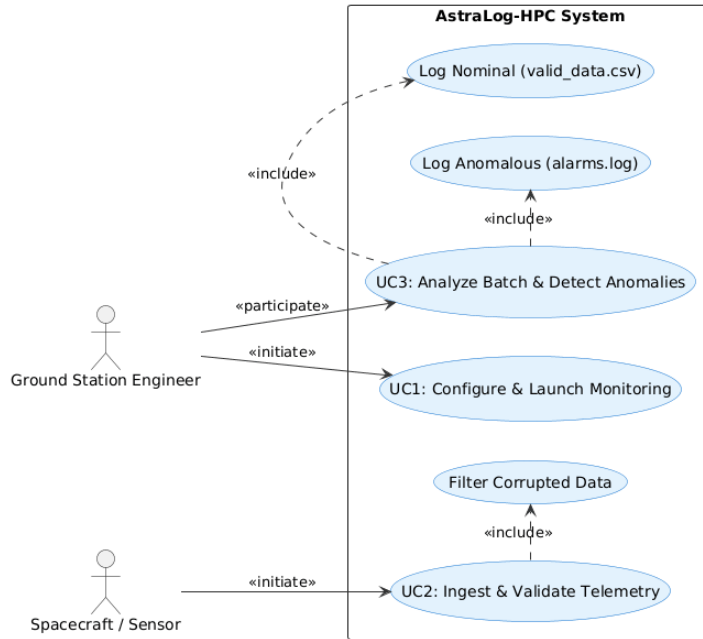


Figure 1: UML Use Case Diagram of AstraLog-HPC

2.3 Domain assumptions

2.3.1 Configuration and Metadata

- [D.C1] **File Consistency and Integrity:** It is assumed that the configuration files provided (`sensors.yaml` and `rules.json`) are syntactically correct and internally consistent. Every `sensor_id` referenced in a monitoring rule is assumed to exist in the sensor configuration.
- [D.C2] **Parameter Immutability:** The monitoring parameters, including sensor definitions and rule thresholds, are assumed to remain static for the duration of an analysis session.
- [D.C3] **Unique Identification:** Each physical sensor on the spacecraft is assigned a unique `sensor_id` that remains constant throughout the mission's lifetime.
- [D.C4] **Valid Parameter Input:** It is assumed that the operational environment provides a strictly positive integer for the batch size.

2.3.2 Data Nature and Temporal Logic

- [D.T1] **Unified Synchronous System Clock:** The system operates under the assumption of a unified mission clock. While telemetry may be generated at sub-second intervals, time is treated strictly at a 1s granularity. Sub-second offsets are ignored or rounded to the nearest second.
- [D.T2] **Equally Spaced Measurements:** It is assumed that measurements for all sensors are provided at regular, fixed intervals (e.g., exactly one measurement per second).
- [D.T3] **Perfect Data Alignment:** The system assumes that all sensor streams are perfectly synchronized. The n -th measurement for any sensor corresponds to the exact same timestamp as the n -th measurement for all other sensors.
- [D.T4] **Data Continuity:** It is assumed that the spacecraft produces data continuously without gaps or interruptions for the duration of the telemetry batch.
- [D.T5] **Chronological Ordering:** Telemetry records in the input files are assumed to be pre-sorted in strictly increasing chronological order ($T_n \geq T_{n-1}$).

2.3.3 Execution Environment (HPC)

- [D.E1] **HPC Resource Availability:** The Galileo 100 cluster and the SLURM Job Scheduler are assumed to be operational and capable of allocating the requested computational resources (CPUs and memory) for the software.
- [D.E2] **Parallel Processing Support:** The underlying hardware and operating system are assumed to support multi-threading and concurrent execution, enabling the parallel evaluation of monitoring rules.
- [D.E3] **File System Persistence:** The environment provides a reliable file system where the machine has the necessary permissions to read input telemetry and write output reports.

2.3.4 Universal Standards

- [D.U1] **Universal Time Reference:** All timestamps across all telemetry files and configurations use a unified time reference (e.g., UTC) that is consistent across all spacecraft subsystems.
- [D.U2] **Data Formatting:** The provided input telemetry files are assumed to adhere to a standard, consistent CSV format, while the system will strictly generate semicolon-separated output logs (`valid_data.csv` and `alarms.log`) as defined in the mission specifications.

2.4 Requirements

2.4.1 Functional requirements

System Configuration & Initialization

- FR.1 The system shall read and parse a `sensors.yaml` file to identify valid sensor IDs.
- FR.2 The system shall read and parse a `rules.json` file to establish the monitoring logic (thresholds, counts, and correlations).
- FR.3 The system shall allow the user to specify a batch size representing the total number of telemetry records or the total elapsed time to be processed in a single analysis cycle.

Telemetry Ingestion & Validation

- FR.4 The system shall ingest telemetry records from provided .csv files.
- FR.5 The system shall validate the syntax and schema of each incoming record.
- FR.6 The system shall discard records that contain Malformed JSON (e.g., truncated strings or syntax errors), Schema Errors (e.g., missing mandatory fields), or Type Errors (e.g., non-numerical values in numerical fields) and continue processing the stream.
- FR.7 The system shall accumulate valid telemetry records into a local buffer until the specified batch size is reached.

Monitoring Rule Evaluation

- FR.8 The system shall sort and prioritize rules based on their defined priority level (Low, Medium, High) before evaluation.
- FR.9 Simple Rule: The system shall trigger an alarm if a sensor value v_n violates a static threshold x (e.g., $v_n > x$ or $v_n < x$).
- FR.10 Step-Difference Rule: The system shall trigger an alarm by computing the signed delta between the current value v_n and the previous value v_{n-1} , defined as $\delta_n = v_n - v_{n-1}$, and applying the rule operator directly to this signed value. For example, a rule may check whether $\delta_n > \Delta$ or $\delta_n < -\Delta$, depending on the configured operator.

- FR.11 Stateful Rule: The system shall trigger an alarm only if a specific rule violation persists for N consecutive measurements.
- FR.12 Logical Correlation Rule: The system shall evaluate complex conditions by combining the Boolean outcomes of existing rules using AND or OR logical operators at the same timestamp.

Result Reporting & Logging

- FR.13 The system shall generate a `valid_data.csv` file containing an aggregated summary of every batch where no rule violations were detected.
- FR.14 The system shall generate an `alarms.log` file containing detailed records for every detected anomaly, including: `Timestamp`, `Rule ID`, `Priority Level`, `Violated Sensor ID(s)`, `Current Sensor Value(s)`.
- FR.15 For alarms triggered by Correlation Rules, the system shall explicitly list all sensors and their respective values involved in the parent rules.

2.4.2 Non-functional requirements

- NFR.1 Performance & Parallel Efficiency: The system shall exploit OpenMP-based shared-memory parallelism for CSV parsing, rule evaluation, and output formatting. Parallel efficiency shall be evaluated experimentally using the following formula:

$$\eta = \frac{T_1}{p \cdot T_p}$$

where T_1 is the single-thread execution time, T_p is the execution time using p threads, and p is the number of processing threads.

- NFR.2 Reliability & Robustness: The system shall discard corrupted CSV records without interrupting the processing of valid telemetry records. Corrupted records include malformed CSV rows, missing fields, explicit corruption markers such as `ERR` or `CORRUPT`, and invalid numerical values.
- NFR.3 Verifiability & Formatting: The system shall generate `valid_data.csv` and `alarms.log` according to the required semicolon-separated output formats, supporting verification through deterministic comparison with expected outputs.
- NFR.4 Portability & HPC Deployment: The system shall be buildable as a self-contained Singularity/Apptainer container image and executable on the Galileo 100 cluster through a SLURM job script.

3 Design

3.1 General Description of the Architecture

The implemented architecture is a single-process, modular OpenMP pipeline. The system runs as one C++17 executable, `astralog_processing`, and exploits shared-memory parallelism through OpenMP threads. The logical architectural responsibilities are realized through in-process modules, vectors, memory-mapped files, thread-local buffers, and ordered sequential writes.

Initial Design Component	Current Implementation
Data Ingestor + Validator	<code>csv_parser.hpp</code> : memory-mapped CSV parser, schema/type validation, and OpenMP line parsing.
Configuration Manager	<code>yaml_parser.hpp</code> , <code>rules_engine.hpp</code> , plus rule preparation in <code>timestamp_processor.hpp</code> .
Batch Accumulator	<code>batch_accumulator.hpp</code> : count/time batching, audit files, and timestamp-safe flushing.
Partition Dispatcher	Phase 2 in <code>timestamp_processor.hpp</code> : builds per-sensor reading sequences.
Worker Pool	Phase 3 OpenMP loop across sensors.
Correlation Sequencer	Phase 4 sequential correlation evaluation.
Logging Proxy	<code>output_formatter.hpp</code> plus ordered sequential file writes in <code>main.cpp</code> .

Table 1: Design-to-implementation mapping of AstraLog-HPC logical components.

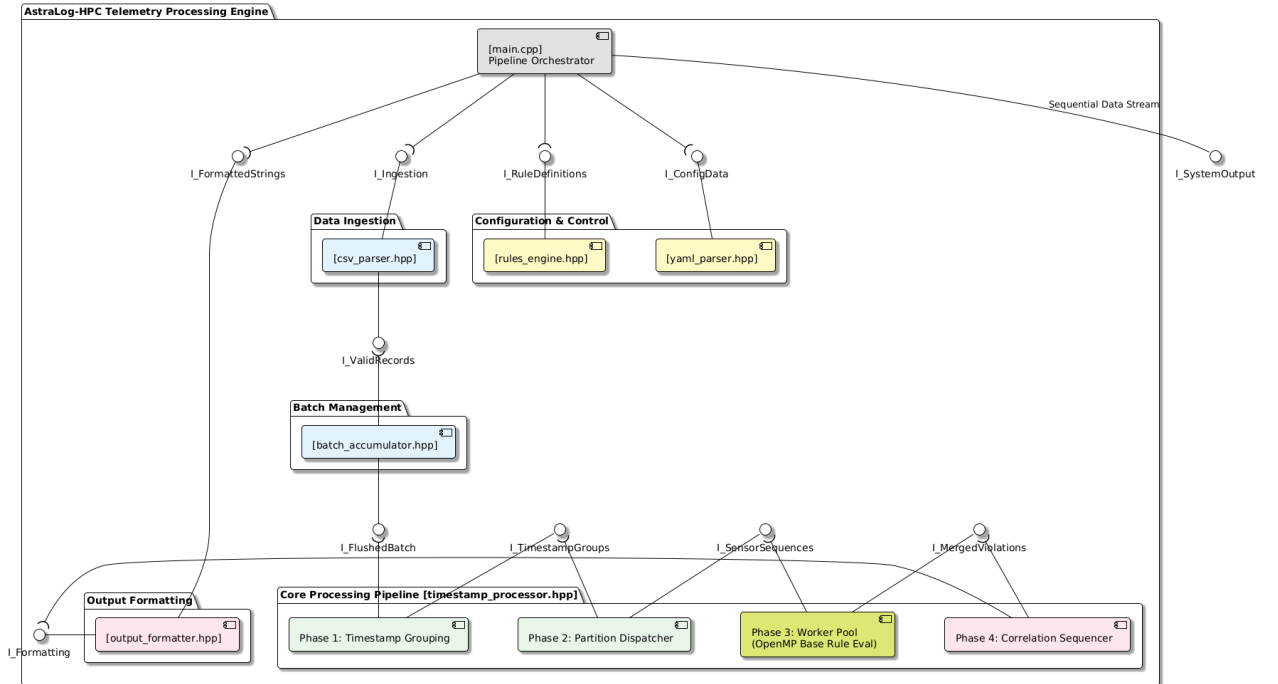


Figure 2: UML Component Diagram

3.1.1 Architectural Components

- **Data Ingestor + Validator:** Implemented by the memory-mapped CSV parser. It reads the input file, builds line offsets, parses rows in parallel with OpenMP, and validates schema/type constraints before records enter the processing pipeline.
- **Configuration Manager:** Loads `sensors.yaml` and `rules.json`, builds the sensor/rule structures, and prepares the rule sets used by the timestamp processor. Rule priority ordering is handled before evaluation so that the runtime phases can focus on computation.
- **Batch Accumulator:** Maintains valid records until the configured count-based or time-based batch boundary is reached. It also manages audit information and preserves timestamp safety when flushing batches.
- **Partition Dispatcher:** Realized as the per-sensor sequence-building phase. It re-organizes in-memory records into sensor-local sequences suitable for independent rule evaluation.
- **Rule Evaluation Worker Pool:** Realized by OpenMP threads inside the same executable. Phase 3 evaluates Simple, Step Difference, and Stateful rules across sensors in parallel, using local data structures to avoid unnecessary synchronization.
- **Correlation Sequencer:** Runs after the per-sensor parallel phase. It evaluates Logical Correlation rules sequentially at each timestamp because those rules depend on the combined outcomes of multiple base rules.
- **Logging Proxy:** Implemented through `output_formatter.hpp` and ordered file writes in `main.cpp`. Parallel formatting can prepare output records, but final writes to `valid_data.csv` and `alarms.log` are serialized to preserve deterministic ordering and strict output formatting.

3.1.2 Architectural Connectors

- **Initialization Interface:** Static configuration data is passed in-process from the configuration parsers to the rule preparation and timestamp processing modules at startup.
- **Memory-Mapped File Access:** Input CSV data is accessed through memory mapping, reducing copy overhead and enabling efficient construction of line offsets for parallel parsing.
- **In-Memory Vectors:** Parsed records, batches, timestamp groups, per-sensor sequences, and violations are exchanged through standard C++ containers rather than external queues or streams.
- **OpenMP Parallel Loops:** Parallelism is expressed through OpenMP loops over independent parsing regions, sensor-local rule evaluation tasks, and output-formatting work. This is shared-memory parallelism inside one process.
- **Thread-Local Violation Buffers:** During parallel evaluation, each thread can accumulate local violations before results are merged. This reduces contention and avoids frequent synchronized writes.

- **Sequential Merge and Ordered File Writing:** After parallel computation, results are merged into timestamp order. The final output stage writes deterministically to `valid_data.csv` and `alarms.log`.

3.2 Sequence diagrams

The implementation-level sequence is organized around the five-phase pipeline in `timestamp_processor.hpp`.

3.2.1 Simple, Step-Difference, and Stateful Rule Evaluation

For each flushed batch, valid records are first grouped by timestamp. The processor then builds per-sensor reading sequences so that each sensor’s chronological values can be evaluated independently. OpenMP distributes these per-sensor sequences across threads. Each thread evaluates the applicable Simple, signed Step-Difference, and Stateful rules and stores detected violations in local or per-timestamp buffers. After the parallel region, violations are merged in deterministic order.

3.2.2 Logical Correlation Rule Evaluation

Correlation rules are evaluated after the base-rule phase. At this point, the system already knows which Simple, Step-Difference, and Stateful rules fired at each timestamp. The Correlation Sequencer scans those timestamp-local outcomes sequentially and applies the configured AND/OR condition. This preserves temporal consistency because correlation rules depend on the combined result set for the same timestamp.

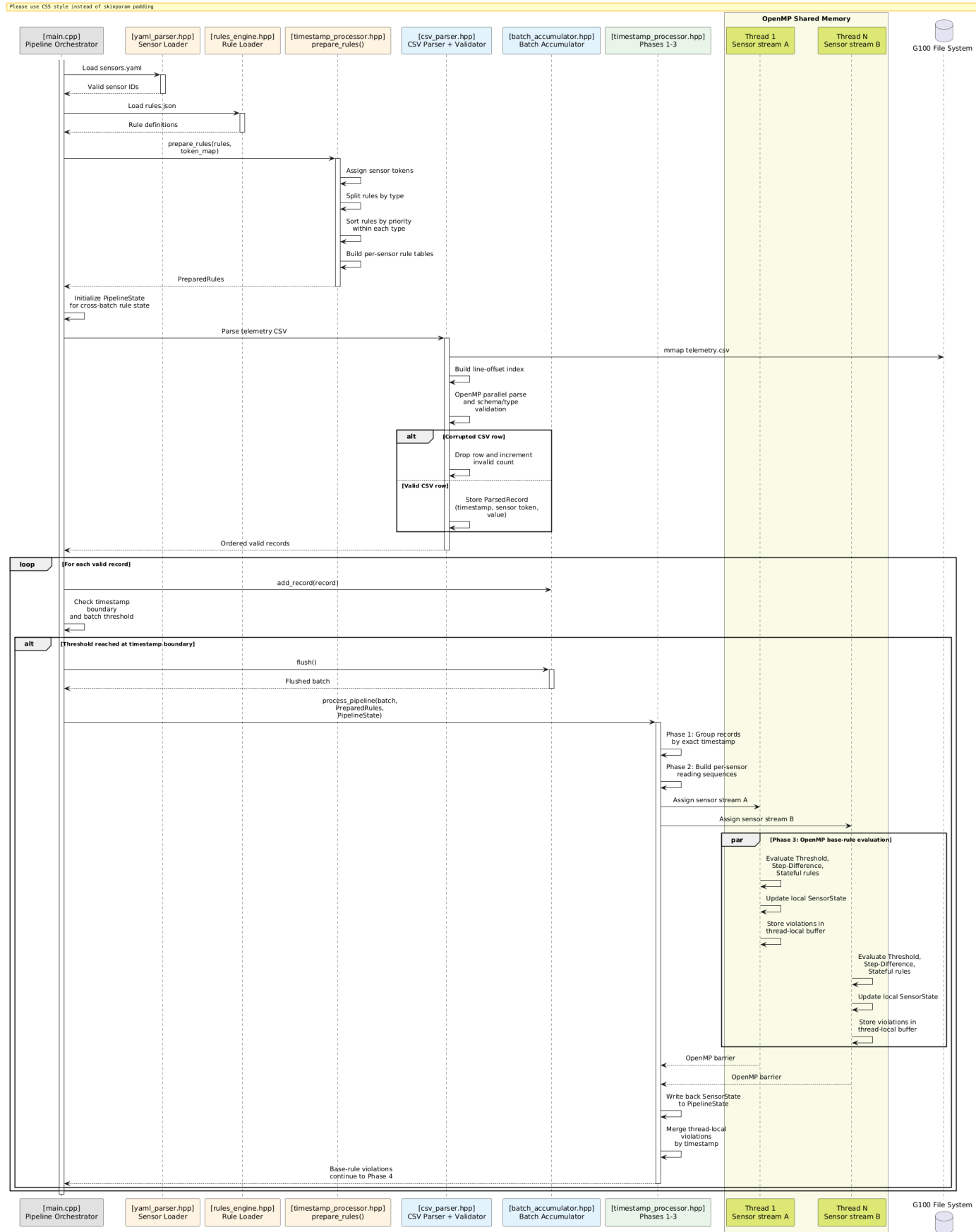


Figure 3: UML Sequence Diagram: Simple, Step-Difference, and Stateful Rule Evaluation

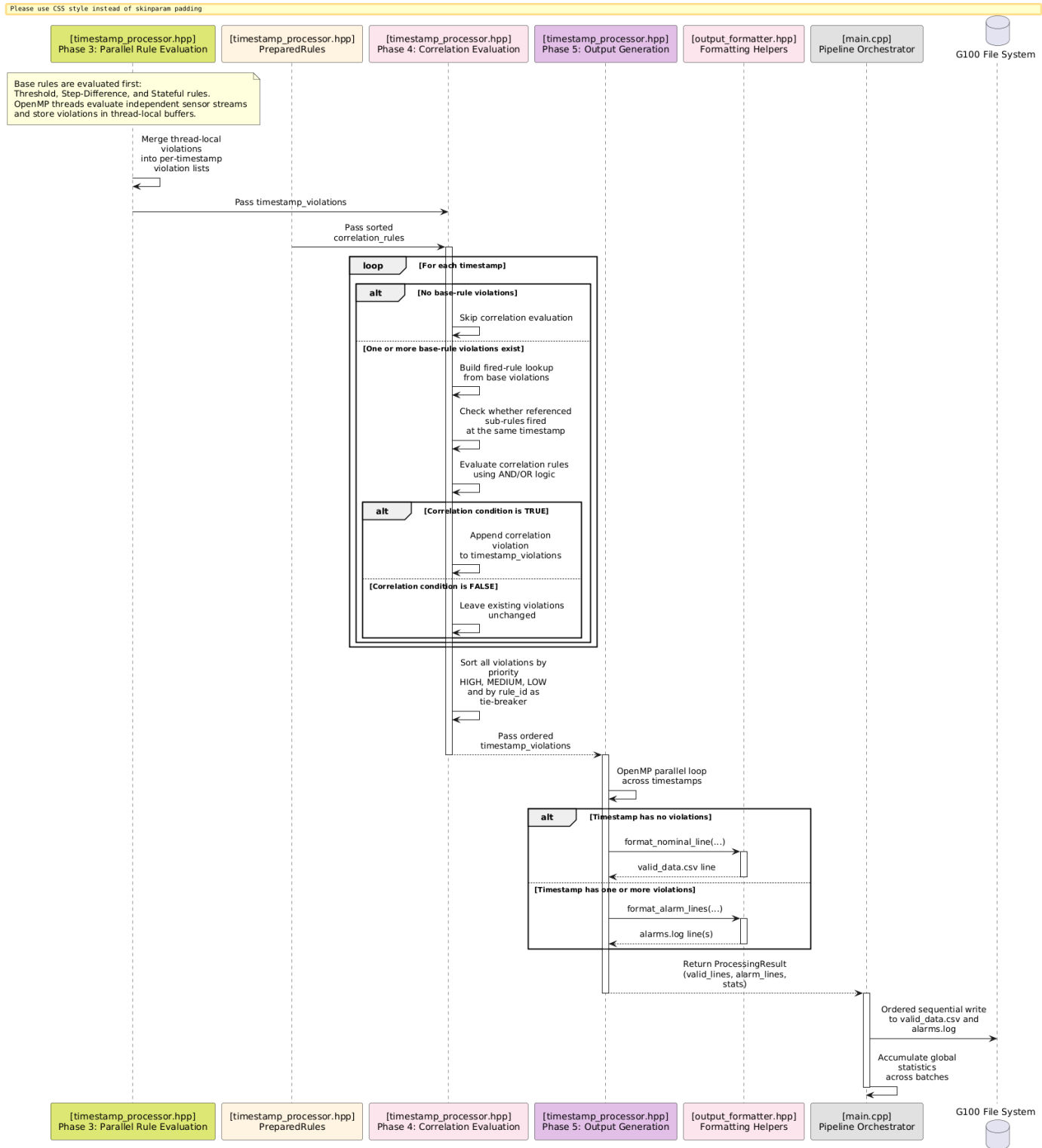


Figure 4: UML Sequence Diagram: Logical Correlation Rule Evaluation

3.3 Critical Points and Design Decisions

3.3.1 Maximizing Performance through Shared-Memory Parallelism

To satisfy the project requirement of improving rule processing performance through parallelization, the implementation applies Divide and Conquer inside a single process. After batching and timestamp grouping, telemetry records are reorganized into per-sensor sequences. These sequences are independent for Simple, Step-Difference, and Stateful rule evaluation, so Phase 3 can distribute them across OpenMP threads without network communication or inter-process dispatching.

This design preserves data parallelism while avoiding the overhead of external scheduling, serialization, socket communication, and queueing middleware. With respect to the previous version of the architectural implementation, the Configuration Manager still prepares and sorts rules by priority before evaluation, but this information is consumed directly by in-process modules rather than sent to remote workers

3.3.2 Handling Correlation after the Parallel Phase

Sensor-based partitioning is efficient for base rules, but Logical Correlation rules require a timestamp-level view of multiple rule outcomes. For this reason, correlation evaluation is intentionally performed after the parallel per-sensor phase. The Correlation Sequencer scans the merged timestamp-local violations and evaluates the configured AND/OR relationships sequentially. This acts as a deterministic synchronization point and preserves temporal consistency without introducing cross-process coordination.

3.3.3 Modularity, Cohesion, and Defensive Boundaries

The memory-mapped CSV parser is responsible for ingestion and validation, the Batch Accumulator manages batch boundaries and audit behavior, the timestamp processor performs grouping and rule evaluation, and the output formatter prepares deterministic output lines. This keeps parsing, batching, rule logic, correlation logic, and output formatting as separate concerns.

Validation is applied at the system boundary. Malformed records, schema errors, and type errors are discarded before valid telemetry records are inserted into the analysis pipeline. The difference from the initial design is that this defensive boundary is implemented through efficient in-process parsing and OpenMP line processing, not through a separate network-facing ingestion service.

3.3.4 Information Hiding and I/O Bottleneck Management

Stateful rule evaluation depends on previous valid measurements and consecutive violation counts. This state is kept inside the rule-processing logic and is not exposed as a shared external service. During parallel evaluation, thread-local or per-timestamp violation buffers reduce contention and hide intermediate data structures from unrelated modules.

The final I/O stage remains deliberately ordered. Although output records can be formatted in parallel, the writes to `valid_data.csv` and `alarms.log` are performed sequentially

to preserve deterministic ordering and the exact semicolon-separated output format required by the specification.

3.4 Design-to-Implementation Deviations

The initial architecture identifies logical components such as dispatcher, worker pool, correlation sequencer, and logging proxy. In the implementation, these are realized inside a single C++17 executable using OpenMP shared-memory parallelism. This preserves the logical responsibilities while avoiding networking overhead and better matching the project's single-node SLURM execution model.

The main deviation is therefore architectural deployment, not functional responsibility. The current implementation should be understood as a logical architecture implemented through a single-process OpenMP pipeline, rather than as a multi-node deployment.

3.5 Deployment Diagram

The deployment topology is a single-node HPC execution. A single Singularity/Apptainer container executes the C++ OpenMP engine on one Galileo 100 compute node. SLURM launches one task, and the task uses 48 OpenMP threads inside the same process. The design does not require a master node dispatching work to multiple containers.

AstraLog-HPC V2.0: Expanded Single-Node HPC Deployment Architecture

Please use CSS style instead of skipparam padding

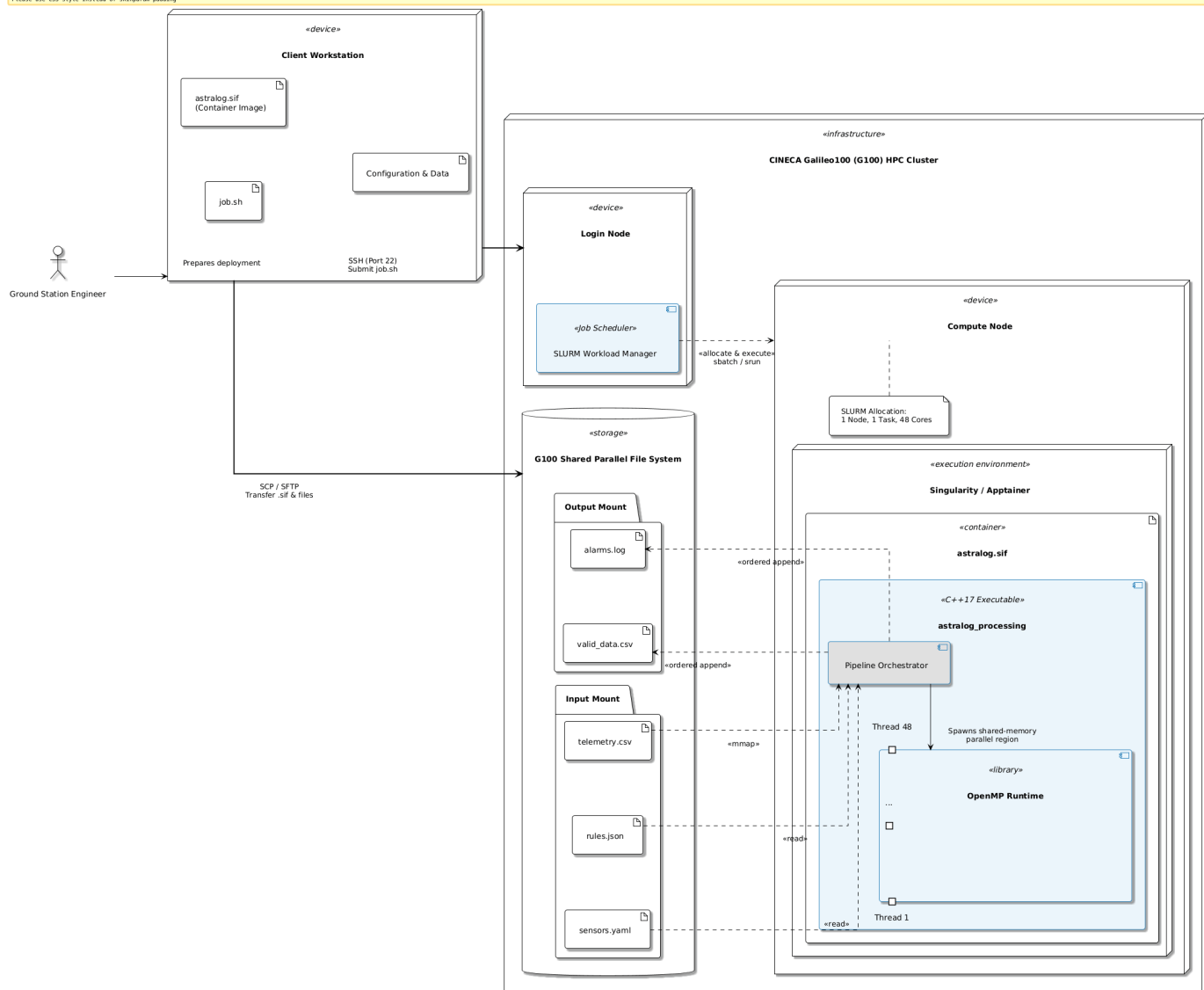


Figure 5: UML Deployment Diagram

4 Work distribution and effort

We comprehensively addressed all project phases and analysed the distribution of effort according to the trend illustrated below. By alternating structured collaborative sessions with focused individual work, we ensured consistent coordination and sustained progress throughout the entire project lifecycle.

Phase	Tasks (Hours)	Total
RASD	Actors (3), Use cases (10), Domain assumptions (3), Requirements (5)	21
DD	Architecture (10), Sequence diagrams (3), Design decisions (5)	18
Total		39

4.1 Personal Contribution and Role in Requirements Analysis and System Design

Antonello Anzalone

"My contribution lies in the combination of curiosity and meticulousness that characterizes my approach, allowing me to explore the connection between a practical case study and the formalism of software engineering used to describe situations and represent solutions. In particular, during the requirements analysis phase, I focused on understanding the concrete meaning of the theoretical tools employed, using them to rigorously model every detail of the system based on the specifications defined by the project group. In the design phase, I worked on identifying inconsistencies between the initial ideas and established software design principles, proposing solutions to address edge cases identified both independently and collaboratively within the team. Finally, I coordinated the overall structure and the writing of the final document, ensuring consistency, clarity, and high formal quality."

Francesco Agosta

"I focused on making sure the design actually worked in practice, not just in theory. I spent a lot of time mapping out the logic through diagrams to keep the team aligned and worked closely across different areas of the project to ensure our choices were technically sound. My main priority was catching those tricky edge cases early on to make sure the final product was solid and reliable. I also stayed focused on the architecture, ensuring it was robust enough to hit all our targets, from functional features to long-term performance."

Gabriele Amodeo

"My primary contribution centered on establishing a comprehensive understanding of the overall problem domain and on the requirements analysis phase. By synthesizing the project's core constraints, I helped shape the high-level architectural design, ensuring our structural decisions aligned with the demands of a single-node OpenMP execution model on an HPC environment."

5 Usage of AI

AI proved to be highly valuable, first and foremost in providing a concrete sense of awareness and depth regarding theoretical and academic tools whose practical usefulness is often not immediately evident.

More specifically, during the requirements analysis phase, AI was used as an additional layer of assurance due to its ability to capture specification details that could easily be overlooked. Its most significant contribution in this phase, however, was its creative support: it helped translate informal and concrete scenarios into highly detailed formal representations, as well as explore more abstract cases with precision. This enabled us to build a clear and well-defined understanding of what the system is and what it is expected to do.

In the design phase, AI was particularly useful in identifying when our initial, rough ideas conflicted with established software design principles. This allowed us to iteratively refine the system architecture, progressively improving our design by leveraging its ability to connect theoretical concepts that initially appeared unrelated but proved to be fundamental when combined.

As a final overarching contribution, AI supported the overall coherence of the project by facilitating alignment between requirements and design choices, ensuring consistency across all stages of development.